

An abstract background on the left side of the slide, featuring a complex network of glowing blue lines and nodes, resembling a digital or molecular structure. The nodes are of various sizes and some are highlighted with a bright blue glow. The lines connect these nodes in a web-like pattern, creating a sense of depth and connectivity.

Mastering in Dart

An in-depth look at Dart's fundamental data structures and programming concepts.

Collections in Dart

Dart provides several built-in collection types to help you manage groups of objects. Each type has unique characteristics regarding order, duplicate values, and access methods.

Order	Ordered (by index)	Unordered	Unordered (keys)
Duplicates allowed?	Yes	No Keys	key ❌, Values ✅
Access	By index (list[0])	No index	By key (map["key"])
Structure	Single values	Unique values	Key-Value pairs

Why Dart uses **Type Inference**?

Type inference means that Dart can automatically figure out (infer) the type of a variable from its value, even if you don't explicitly write the type. This feature helps in writing cleaner and more concise code while maintaining type safety.



Operations in Dart

In Dart, operations are used to perform actions on values and variables. They are divided into several categories:

1. Arithmetic Operations

These are used for mathematical calculations.

- + Addition
- - Subtraction
- * Multiplication
- / Division (returns a double)
- ~/ Integer Division (returns an integer)
- % Modulus (remainder)

2. Relational (Comparison) Operations

Used to compare two values. The result is always true or false.

- == Equal
- != Not equal
- > Greater than
- < Less than
- >= Greater than or equal
- <= Less than or equal

3. Logical Operations

Used to combine conditions.

- && Logical AND
- || Logical OR
- ! Logical NOT

Operations in Dart (cont.)

4. Assignment Operations

Used to assign values to variables.

- = Simple assignment
- += Add and assign
- -= Subtract and assign
- *= Multiply and assign
- /= Divide and assign

Example:

```
void main() {  
  int n = 10; n += 5;  
  // n = n + 5  
  print(n);  
}
```

5. Unary Operations

Used on a single operand.

- ++ Increment
- -- Decrement
- - Negation (change sign)

Example:

```
void main() {  
  int a = 5; print(-a);  
  // -5  
}
```

6. Type Test Operations

Used to check the type of a variable.

- is Returns true if object is of a specific type
- is! Returns true if object is NOT of that type

Example:

```
void main() {  
  String name = "Dart"; print(name is  
  String);\true  
  print(name is! int); \true  
}
```

Error Handling in Dart

1

Try-Catch-Finally

Dart uses **try-catch-finally** to handle errors safely.

- **try** → contains the code that might throw an error.
- **catch** → handles the error if it occurs.
- **finally** → always runs, whether there was an error or not (useful for cleanup).

Example:

```
void main() {  
  try { int result = 10 ~/ 0; print(result); } catch (e)  
  {print("Error: $e"); }  
  finally { print("This always runs."); }  
}
```

Output:

```
Error: IntegerDivisionByZeroException This always  
runs.
```

2

Throwing Exceptions

You can create and throw your own exceptions using **throw**.

Example:

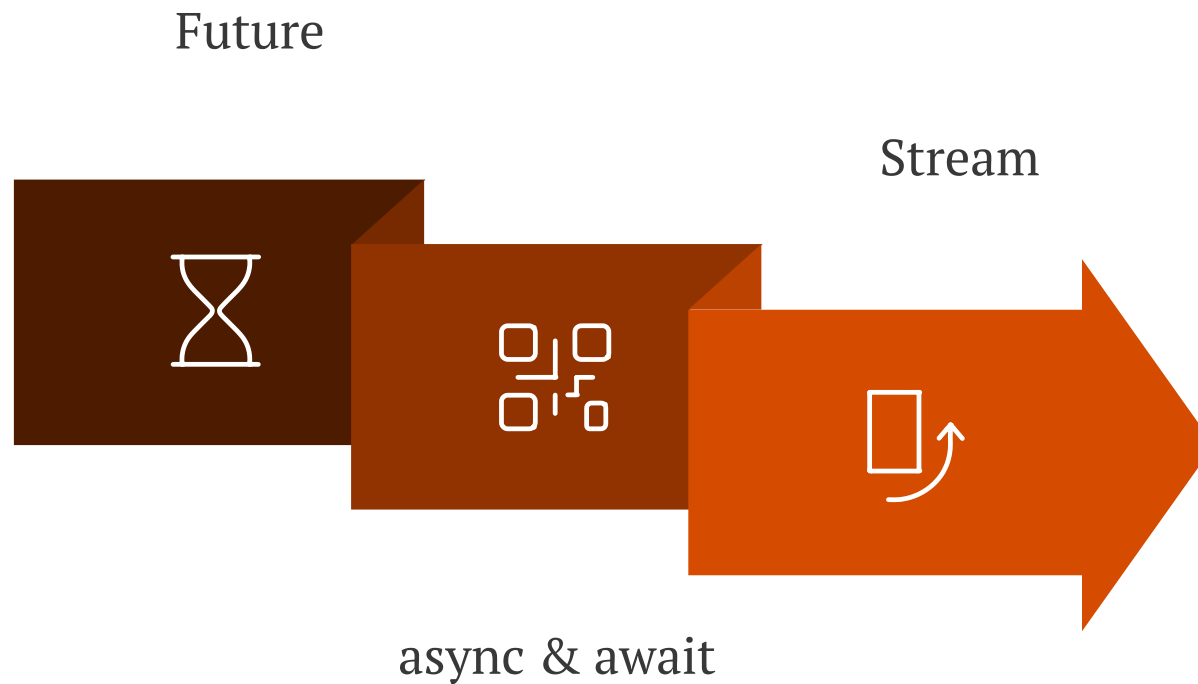
```
void checkAge(int age) {  
  if (age < 18) {  
    throw Exception("You must be 18 or older!"); }  
  }  
  void main() {  
    try {checkAge(15);}  
    catch(e){print("Caught an exception: $e"); }  
  }
```

Output:

```
Caught an exception: Exception: You must be 18 or  
older!
```

Asynchronous Programming

Sometimes we need to perform tasks that take time (like loading data from the internet). If we run them normally, the program will stop and wait. But with **async / await / Future / Stream**, the program keeps running smoothly.



Asynchronous Programming (cont.)

1. Future

A **Future** represents a value that will be available in the future (not immediately).

Example:

```
Future<String> fetchData() {  
    return Future.delayed(Duration(seconds: 2),  
    (){ return "Data loaded!"; });  
}  
  
void main(){  
    fetchData().then((data) { print(data); });  
}
```

2. async & await

- **async** makes a function asynchronous.
- **await** tells the program to wait for the Future result before continuing.

Example:

```
Future<String> fetchData() async {  
    await Future.delayed(Duration(seconds: 2)); return "Data  
loaded with async/await!"; }  
  
void main() async {  
    print("Loading...");  
    String result = await fetchData(); print(result);  
}
```


Asynchronous Programming (cont.)

3. Stream

A **Stream** handles data that arrives in parts (like chat messages or sensor readings).

Example:

```
Stream<int> countStream(int max) async* {  
  for (int i = 1; i <= max; i++){  
    await Future.delayed(Duration(seconds: 1));  
    yield i;}  
}  
  
void main() async {  
  await for (var number in countStream(5))  
  {print(number);}  
}
```

yield vs return

return

- Returns **a single value only**.
- Ends the function immediately.
- Used with **Future** functions.

Example:

```
Future getMessage() async { return "Hello";}  
void main() async { print(await getMessage()); }
```

yield

- Returns **multiple values, one by one**.
- Keeps the function **alive** until it finishes sending all values.
- Used with **async*** → creates a **Stream**.

Example:

```
Stream numbers() async* { yield 1; yield 2; yield 3; }  
void main()async{  
  await for (var n in numbers()) { print(n); } }
```

Preparation for Flutter

Packages & Libraries

To use external or built-in libraries, we use import.

Example:

```
import 'dart:math'; void main() { print(sqrt(16)); }
```

```
import 'dart:io';  
void main() { print("Enter your name:"); String? name = stdin.readLineSync(); }
```